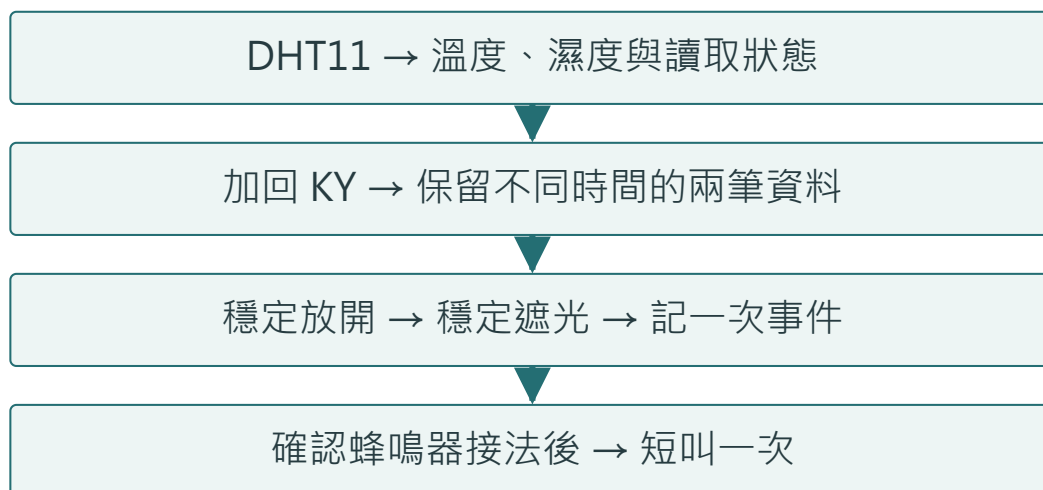


做一個環境紀錄與遮光提醒器

先讓感測器回報，再讓程式決定何時提醒。

上週我們讓 ESP32 讀到光線變化。這週加上溫濕度，並處理「讀不到」與「只晃動一下」的情況。



今天做完後，你可以遮住光敏模組，看到遮光次數增加。符合接線條件時，蜂鳴器也會短叫一次。手一直蓋著，不應一直增加次數。

電腦會顯示光線、溫濕度、時間和失敗原因，我們再把輸出存進筆記。本週先用 USB 看紀錄，還不需要 Wi-Fi 或雲端。

換接線前先拔 USB，依圖核對每條線的兩端。光敏、DHT 與蜂鳴器各使用不同的訊號 GPIO。

不用從 PDF 一頁一頁抄程式

1. 已有教師提供的完整 IoT 資料夾，就直接使用同版本資料夾，不重新下載另一版。
2. 沒有資料夾時，開啟 [課程 GitHub](#)，按 **Code** → **Download ZIP**。下載完成後在檔案總管右鍵 ZIP，選「全部解壓縮」。
3. 打開解壓縮後的資料夾，再開 **IOT_Introduction**。不要直接在 ZIP 預覽視窗操作。
4. 依下表找到當段程式；在 Arduino IDE 選「檔案 → 開啟」，選該資料夾中的同名 **.ino**。瀏覽器裡看到程式文字，不等於已在 IDE 開啟。
5. 修改前選「檔案 → 另存新檔」，存進自己的練習資料夾。按 **Ctrl+F** 搜尋講義指定的設定名稱，改等號右邊的值，保留分號，再按 **Ctrl+S**。

本週依序開這些檔案

以下從 IOT_Introduction 資料夾開始找；每次只開當段要用的檔案。

- `examples/week04_dht11_quality/week04_dht11_quality.ino`
- `examples/week04_dual_sensor_alarm/week04_dual_sensor_alarm.ino`
- `docs/teaching_drafts/week4_redesign/button_environment_log/button_environment_log.ino`

換另一支程式時，原來修改的設定**不會自動帶過去**。依該段設定表逐項填寫；不把短片段當成完整程式。

若下載內容和講義列出的檔名不一致，先取得相符的課程資料夾，不猜替代檔。開啟檔案時，ESP32 的 USB 與外部電源保持拔除。

今天拿這些就好

電池、馬達、舵機先留在盒子裡。

器材	今天用來做什麼
ESP32-S3、可傳資料的 USB 線、電腦	執行程式、顯示紀錄
麵包板、公對公、公對母、母對母線	固定測點與連接模組
電表、220 Ω 與 330 Ω 各一顆	看懂阻值與量程
KY-018、DHT11 三線模組	讀光線、溫度、濕度
已核對的蜂鳴器；必要時另取 1 k Ω	最後的短音提示
上週同一組器材的光線紀錄	沿用室內光、遮光各自的最小與最大值

1. 關掉外接電源，拔 USB，確認 PWR 熄滅。
2. 拍照保存原來作品，再拆下本次不用的線。
3. ESP32 放在桌面，不整片插進這塊 400 孔麵包板。
4. 本週用麵包板左半部 a~e。第 3、6、15 列先留空。

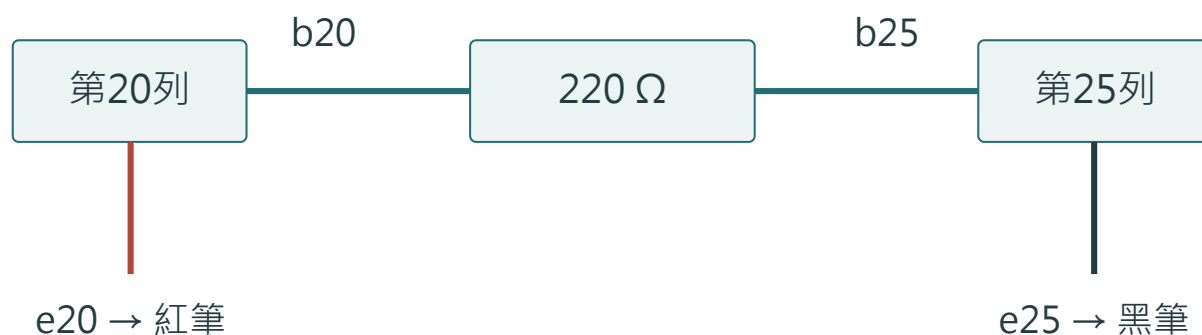
DHT11 的三針順序不能靠線色猜。教師需在課前提供本批模組的已確認腳序與程式腳位；若沒有這份資訊，先做電阻及紙上判讀，不接未知模組。蜂鳴器未確認時，仍可完成無聲的紀錄與事件作品。

電阻不用接 ESP32

先把它固定好，再拿表筆。

1. USB 保持拔除；這個測試區不接任何電源或模組。
2. 把 $220\ \Omega$ 電阻的兩腳分別插在 b20、b25。
3. 公對公線插 e20、e25。兩個自由端分開固定。
4. 電表黑筆插 COM，紅筆插 $V\Omega$ 或標有 Ω 的共用孔，不插 10A。
5. 選 Ω 功能及可涵蓋 $330\ \Omega$ 的量程。紅筆碰 e20 的自由端，黑筆碰 e25 的自由端。

外部斷電，測試區不接 ESP32



每列 a~e 相通；第20列與第25列不相通。

典型手動表可用 $2k\Omega$ 。若手上表的 2k 位置與二極體功能不易區分，先使用已確認的 $20k\Omega$ 檔。自動表選 Ω ，另記畫面的單位。

不要把粗表筆塞進麵包板，不用手指同時捏住兩端金屬。量電阻時，電表自己提供小測試電流；不需要外部供電。

換檔不會把電阻變大

比較同一顆、同一對測點，只改量程。

先預測：標稱 $220\ \Omega$ ，在約 $200\ \Omega$ 的範圍內量得下嗎？

1. 保存合適 Ω 量程下的完整讀值，包括小數點與單位。
2. 移開表筆，改 $200\ \Omega$ 範圍，再碰同一對測點。
3. 移開表筆，回到原本合適量程，確認讀值回來。
4. 表筆移開、電表 OFF，改放 $330\ \Omega$ ，重複比較。

教學示例，非本次實測	怎麼讀
20k Ω 檔顯示 0.22 k Ω	$0.22 \times 1000 = 220\ \Omega$
200 Ω 檔顯示最左側單獨 1 或 OL	可能超量程；不是 1 Ω
表筆分開，也顯示 1 或 OL	可能沒有接通測試路徑

合適檔有讀值、較小檔超量程，才支持「量程太小」的解釋。兩檔都沒讀值，先查是否碰到正確兩端。

自動量程表不能手動換檔時，實際量兩顆，再用表內示例解釋量程。不用另買電表。

完成後取下電阻與測試線，分別標示收好，電表 OFF。

同列五孔會繞過電阻

b20 與 e20 內部相通，b25 與 e25 內部相通。

正確量測路徑：

紅筆 → e20 → b20 → 電阻 → b25 → e25 → 黑筆

若電阻兩腳都放在第 20 列，同列金屬片就提供另一條低阻路徑。你量到接近 0，不代表電阻本體真的變成 $0\ \Omega$ 。

這和上週量電壓不同：量電阻要斷開外部電源；量電壓則是讀通電時兩點的電位差。

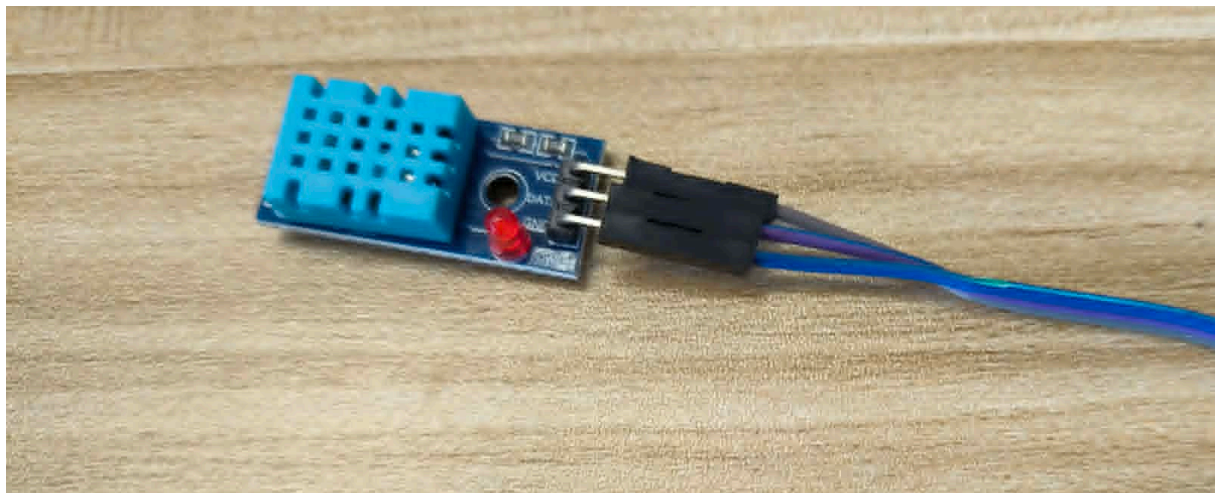
量完保存	自己填，不抄示例
220 Ω	檔位 _ _ / 完整顯示 _ _ / 換算 _ _ Ω
330 Ω	檔位 _ _ / 完整顯示 _ _ / 換算 _ _ Ω
過小量程或畫面判讀	提示 _ _ / 我的解釋 _ _

330 Ω 量成 326 Ω ，不必立即判定壞掉。標稱阻值有容差，電表與接觸也有誤差。色環與容差的補充放在第 59 頁。

下一步換成感測器；剛才的 220、330 Ω 不直接加到 DHT11 接線裡。

這塊藍色的是 DHT11

它把溫度與相對濕度，用資料訊號傳給 ESP32。



已購 DHT11 模組的實物照片。接頭遮住部分標示；這張照片不能代替完整腳序確認。

功能名稱	接到哪種地方
VCC：模組電源	本課已確認可用的 3.3 V 電源
GND：共同參考與回路	ESP32 的 GND
DATA：傳送讀數的腳	本課設定的 DHT 專用 GPIO

三線模組和裸裝四腳 DHT11 不一樣，網路上的四腳圖不能直接套用。DATA 也不是光敏模組的 S 電壓。

先在自己的模組照片上指出 VCC、DATA、GND，對照課堂提供的已確認接法。不能從「紅線應該是電源」反推腳位。

目前資料仍缺：斷電取下三線接頭後的排針絲印照片、這片模組的 3.3 V 供電 / DATA 電位確認，以及本組 PIN_DHT。尚未取得時，到下一段只做裸板編譯與 blocked 訊息辨認，不接 DHT，也不把後面溫濕度示例當成自己的讀值。

先準備程式，還不接模組

今天仍使用 Arduino IDE 和 Serial Monitor。

1. 打開 Arduino IDE。若停在上一週的程式，另開本週的 [DHT11 程式](#)。下載時保留 .ino 檔與同名資料夾。
2. 若 IDE 問是否建立同名資料夾，選建立。完整程式也在本講義後面，不是只貼下一頁的設定幾行。
3. ESP32 暫不接杜邦線。USB 線插在這塊板背面標 **COM** 的接頭，再接電腦。
4. 依上週這塊板成功使用的設定，選 **ESP32S3 Dev Module** 與這次實際出現的連接埠。
5. 這條 CH343 / UART 連線使用 **USB CDC On Boot: Disabled**。不要直接照抄別人的 COM8。

Serial Monitor 是 Arduino IDE 的文字觀察窗。稍後可由「工具 → 序列埠監控視窗」開啟，速度選 **115200**。

本頁針對已辨認的 YD-ESP32-S3 Type-A V1.5 / CH343 板。若實物不同，先依自己的板卡文件確認 USB 路徑與設定，不猜接頭或 GPIO 位置。

讓程式看得懂 DHT11

程式庫是別人已寫好的讀取方法，不是新的硬體。

1. Arduino IDE 左側打開 **Library Manager** / 程式庫管理員。
2. 搜尋 **DHT sensor library**，選作者 **Adafruit** 的版本。
3. 安裝本教材對照使用的 **1.4.7**；依提示安裝 **Adafruit Unified Sensor** 相依套件。
4. 本教材既有編譯紀錄使用 Unified Sensor **1.1.15**、Arduino-ESP32 **3.3.11**。安裝版本不同，要另記版本，不假裝完全相同。
5. 按左上角 勾號 **Verify**，先確認程式能編譯。

```
#include <DHT.h>
```

這行的意思是使用 DHT 程式庫。若出現 `DHT.h: No such file or directory`，先檢查程式庫是否安裝完成，不改接線。

勾號是編譯，右箭頭才是上傳。編譯成功只代表電腦能產生程式，不代表感測器已接對或讀取成功。

只改開頭的三個設定

腳位要和本批實物接法一致。

程式中搜尋的名稱	這次填什麼
PIN_DHT	課堂已確認、未與其他功能共用的 GPIO 編號
MODULE_PROFILE_CONFIRMED	本批腳序、3.3 V 供電與 DATA 電位均確認後才填 <code>true</code>
DEVICE_ID	自己的組別，例如 "group01"，保留雙引號

原始檔保留 `PIN_DHT = -1` 與確認值 `false`。這是避免在未知接法上開始讀取，不是要把線接到「負一號腳」。

1. 填好課堂設定，再按 **Verify**。
2. 板上仍不接感測器，按 **右箭頭 Upload**。
3. 上傳完成後，關掉 Serial Monitor，拔 USB，確認 PWR 熄滅，才接下一頁的線。

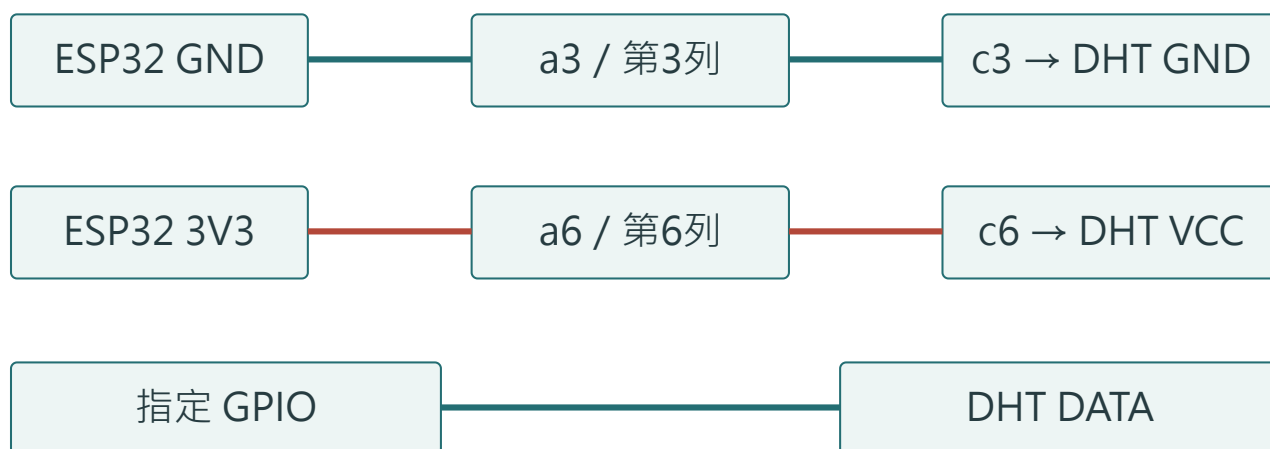
如果還沒有確認資料，保留預設值。程式會停止感測操作並提示設定未完成；不要為了消除提示隨便填 `true`。本課不把 DHT 模組改接 5 V，也不把 5 V DATA 接進 GPIO。

DHT11 先單獨工作

以下按功能接線，不表示模組三針的左右順序。

USB 拔除，KY 與蜂鳴器都還沒接。

線的起點	終點	線型
ESP32 GND	a3	公對母
ESP32 3V3	a6	公對母
DHT 已確認的 GND	c3	公對母
DHT 已確認的 VCC	c6	公對母
DHT 已確認的 DATA	PIN_DHT 指定的 GPIO	母對母



三條功能路徑；不是模組排針的實際排列。

模組放在板外，用母端套住排針。第 3 列與第 6 列不能直接互接。第 15 列目前留空。

通電前逐條核對起點與終點，尤其是模組 VCC / GND。有裸露金屬互碰、腳位辨識不清或接法不同，先保持斷電。

接 USB，等數字出現

不要一秒內沒有畫面就開始拔線。

1. 檢查沒有短接後，把 USB 接回 **COM** 接頭。
2. 打開 Serial Monitor，選 **115200**。
3. 若錯過開頭提示，按一次板上 **RST / Reset**。序號與開機時間會重新開始。
4. 約每 **2.5 秒** 讀一次，先保存至少 **10 筆** 實際讀取嘗試；有失敗也留下。

以下為**教學示例**，分行方便閱讀，不是你應該量到的指定溫度：

```
sensor=dht11 source=hardware sample=1
temperature_c=25.0 humidity_pct=50.0
valid=true quality=usable
reason=basic_checks_passed
```

你的程式會印更多欄位，通常在同一行。選取完整輸出、複製貼到課堂筆記，另寫操作時間；Monitor 不會自動替你保存檔案。不要為了符合示例改動實測數字。

如果看到 `read_failed`，先保存畫面，再依第 58 頁排查。如果是設定未完成，回第 10 頁，不從感測線下手。

week04_dht11_quality.ino

第 1 ~ 32 行。

```
#include <DHT.h>
#include <math.h>

// Do not set a GPIO or enable this gate until the exact module is approved.
const int PIN_DHT = -1;
const bool MODULE_PROFILE_CONFIRMED = false;
const char* DEVICE_ID = "CHANGE_ME";
const unsigned long READ_INTERVAL_MS = 2500;
// Classroom warning policy, NOT the DHT11 measurement/operating specification.
const float ROOM_MIN_C = 10.0f;
const float ROOM_MAX_C = 40.0f;
const float JUMP_C = 5.0f;
const float JUMP_RH_POINTS = 10.0f;

DHT dht(PIN_DHT, DHT11);
bool injectFailure = false;
bool havePrevious = false;
float previousC = 0.0f;
float previousRh = 0.0f;
unsigned long previousMs = 0;
unsigned long lastReadMs = 0;
unsigned long sampleNumber = 0;

bool profileReady() {
    return PIN_DHT >= 0 && MODULE_PROFILE_CONFIRMED;
}

void reportReading(float temperatureC, float humidityPct, unsigned long now, bool
injected) {
    const char* quality = "usable";
    const char* reason = "basic_checks_passed";
    bool valid = true;
    // Missing is not zero. Never substitute the previous value for this row.
```

同一份 .ino 檔案；分頁不代表另開程式。

week04_dht11_quality.ino

第 33 ~ 61 行。

```
if (!isfinite(temperatureC) || !isfinite(humidityPct)) {
    valid = false; quality = "invalid";
    reason = injected ? "injected_read_failed" : "read_failed";
} else if (humidityPct < 0.0f || humidityPct > 100.0f) {
    valid = false; quality = "invalid"; reason = "rh_out_of_bounds";
} else if (temperatureC < ROOM_MIN_C || temperatureC > ROOM_MAX_C) {
    quality = "suspect"; reason = "outside_classroom_policy";
} else if (havePrevious && now - previousMs <= 2 * READ_INTERVAL_MS &&
    (fabsf(temperatureC - previousC) > JUMP_C ||
    fabsf(humidityPct - previousRh) > JUMP_RH_POINTS)) {
    quality = "suspect"; reason = "abrupt_change";
}
Serial.printf("device=%s sensor=dht11 source=%s sample=%lu uptime_ms=%lu "
    "temperature_c=%.1f humidity_pct=%.1f valid=%s quality=%s reason=%s\n",
    DEVICE_ID, injected ? "injected" : "hardware", sampleNumber, now,
    temperatureC, humidityPct, valid ? "true" : "false", quality, reason);
// Only adjacent real finite/basic-range records enter the jump comparison.
havePrevious = valid && !injected;
if (havePrevious) {
    previousC = temperatureC; previousRh = humidityPct; previousMs = now;
}
}

void setup() {
    Serial.begin(115200);
    delay(500);
    if (!profileReady()) {
        Serial.println("week=4 sensor=dht11 status=blocked
reason=module_or_gpio_profile_missing");
        return; // No dht.begin(), pinMode(), or sensor transaction before approval.
    }
}
```

同一份 .ino 檔案；分頁不代表另開程式。

week04_dht11_quality.ino

第 62 ~ 83 行。

```
}
dht.begin();
lastReadMs = millis(); // Give the powered sensor an initial 2.5-second wait.
Serial.printf("week=4 sensor=dht11 status=ready device=%s pin_dht=%d
interval_ms=%lu\n",
              DEVICE_ID, PIN_DHT, READ_INTERVAL_MS);
Serial.println("commands=f:inject_missing r:resume_hardware");
}

void loop() {
  if (!profileReady()) return;
  if (Serial.available()) {
    char command = Serial.read();
    if (command == 'f' || command == 'r') {
      injectFailure = command == 'f';
      havePrevious = false;
      lastReadMs = millis(); // Mode change also waits before the next sample.
      Serial.printf("event=mode source=%s next_read_after_ms=%lu\n",
                    injectFailure ? "injected" : "hardware", READ_INTERVAL_MS);
    } else if (command != '\r' && command != '\n' && command != ' ') {
      Serial.println("event=command status=rejected reason=unknown_command");
    }
  }
}
```

同一份 .ino 檔案；分頁不代表另開程式。

week04_dht11_quality.ino

第 84 ~ 94 行。

```
unsigned long now = millis();
if (now - lastReadMs < READ_INTERVAL_MS) return;
lastReadMs = now;
sampleNumber++;
float temperatureC = NAN, humidityPct = NAN;
if (!injectFailure) {
    humidityPct = dht.readHumidity();
    temperatureC = dht.readTemperature(); // Same cached frame, not another 2.5 s
cycle.
}
reportReading(temperatureC, humidityPct, now, injectFailure);
}
```

同一份 .ino 檔案；分頁不代表另開程式。

25.0 和 50.0 分別是什麼？

每個數字都要和欄位、單位一起看。

欄位	例子怎麼讀
temperature_c=25.0	攝氏 25.0 度；不是 25 V
humidity_pct=50.0	相對濕度 50%；不是空氣一半是水
sample=1	本次開機的第 1 次讀取嘗試，失敗也計次
uptime_ms	本次開機以來的毫秒；不是時鐘日期
source=hardware	這次走實際感測器讀取，不表示讀取一定成功

相對濕度，是當時空氣中的水氣，相對於同溫度下飽和狀態的程度。溫度改變時，相對濕度也可能改變。

先看 10 筆的序號和時間是否前進，再看數值。房間穩定時，連續幾筆相同不等於程式卡住。

10 筆之間有 9 個間隔，約需 22.5 秒，另加第一次等待與操作時間。看筆數完成，不靠秒錶硬湊。

顯示一位小數，不等於感測器精度是 0.1°C。今天先練習取得可追溯紀錄，不把 DHT11 當校準過的精密儀器。

DATA 傳的不是「溫度電壓」

供電路徑與資訊路徑分開看。

供電回路 (簡化)



資訊流



ESP32 → UART → CH343 → USB → Monitor

上半部是電源回路：模組需要電源才能工作，GND 也提供共同參考。沒有畫出模組內所有電路。

下半部是資訊流：DHT11 用有時間規則的數位訊號傳資料，程式庫把它解讀成溫度與濕度。

因此，不用 `analogRead(DATA)` 取得溫度，也不能把電表量到的 DATA 平均電壓當作攝氏度。

DHT11 不適合每 50 ms 要求一次新測量。本例每 2500 ms 讀取；程式庫也有最小間隔與暫存資料的處理。

只少 DATA，電源還在

資訊傳遞示意；僅作圖上推演，保持現有實物接線不動。

正常的連接 / 處理



只改標記的地方



箭頭表示資訊處理順序，不是供電或電流路徑。

想一想： VCC、GND 仍連著，只假設 DATA 不通。下一次真正嘗試讀取時，應把上一筆 25°C 當成新值嗎？

原理提示： 有供電不代表能取得新資料。讀取失敗要留下失敗狀態；舊數值不是這次成功的量測。

只有標記處改變，其餘供電、程式與環境條件沿用正常情境。不可把未知結果直接寫成 0、LOW 或「一定停止」。

在輸入框送出 f，再送 r

用程式模擬失敗，不在通電時拔感測器。

1. 先找到剛才紀錄中至少 3 筆正常資料，不必重收一套。
2. 在 Monitor 上方可輸入文字的一行點一下，打小寫 **f**，按 Enter；不是在下方滾動的輸出區打字。先確認後續出現 `source=injected`，再做下一步。
3. 留下 3 筆 `source=injected` 的失敗紀錄。
4. 打小寫 **r**，送出。等下一次取樣，留下 3 筆恢復後紀錄。

操作	應觀察的變化
f	不向 DHT 取新值，刻意產生 NaN； <code>valid=false</code>
r	恢復向 DHT 讀取；成功與否仍要看後面的資料

NaN 表示這裡沒有可用的數值，**不是 0°C，也不是 0%**。

模式改變後會重新等待約 2.5 秒，不會立刻拿上一筆填成新資料。f / r 不會像 Reset 那樣把 sample 歸零。

這次測到的是「程式收到無效資料時如何處理」。它不是斷線實測，也不能算進真實感測器故障率。如果原本就讀不到，先記錄原始失敗，不假裝已完成正常 → 失敗 → 恢復。

有數字，不一定適合拿來判斷

`valid` 看能否當數值使用，`quality` 說明本課規則的檢查結果。

檢查順序	程式給的結果
1. 溫度或濕度不是有限數值	<code>valid=false</code> 、 <code>invalid</code>
2. 濕度小於 0% 或大於 100%	<code>valid=false</code> 、 <code>invalid</code>
3. 溫度低於 10°C 或高於 40°C	<code>valid=true</code> 、 <code>suspect</code>
4. 距前一筆不超過 5 秒，溫差 >5°C 或濕度差 >10 個百分點	<code>valid=true</code> 、 <code>suspect</code>
5. 以上都沒有	<code>valid=true</code> 、 <code>usable</code>

10~40°C 是本課室內活動的檢查範圍，不是 DHT11 的完整規格。超出範圍先查原因，不改寫成感測器一定壞了。

`usable` 只表示通過這幾項檢查，不保證讀值精準。程式只記第一個符合的原因，不會把所有問題一次列出。

先保留原數值、時間、品質及原因。不能把無效值改成上一筆正常溫度，再假裝它是新測量。

哪一筆要先查原因？

以下都是教學假資料，各題獨立，不是連續五筆。

每題前一筆都是 25°C、50% RH，距現在 2.5 秒。依上一頁規則填 `valid`、`quality` 與原因。

題目	新溫度	新濕度	你的判斷
A	NaN	50%	--
B	25°C	120%	--
C	45°C	50%	--
D	31°C	50%	--
E	30°C	60%	--

再想兩件事：

1. C 同時出現範圍與變動問題，程式會先記哪一個？
2. E 的變化剛好是 5°C、10 個百分點，是否超過程式門檻？

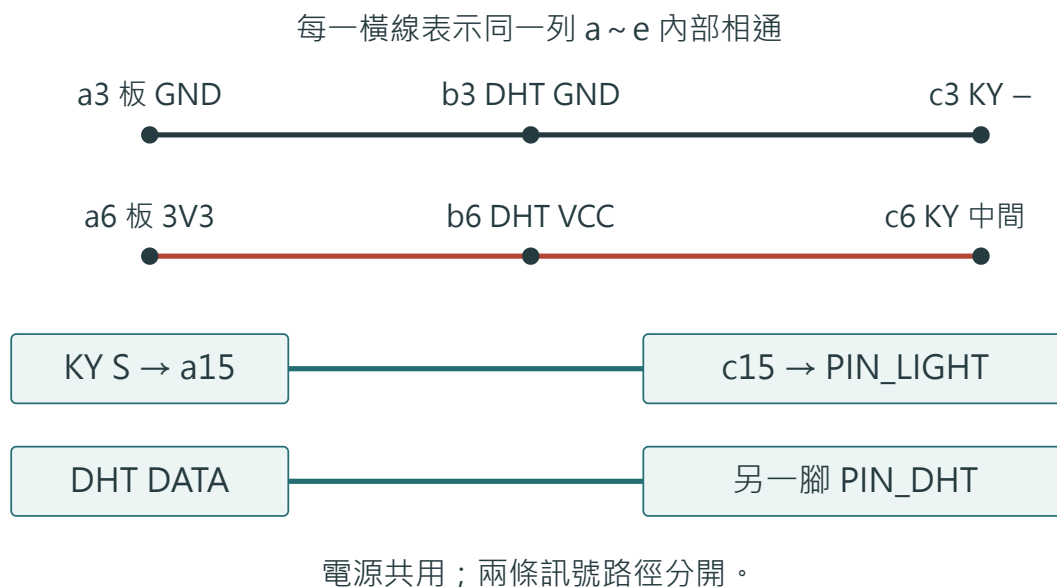
判斷每筆資料是否有效，並指出欄位與原因。

兩個模組，共用電源但不共用訊號

關閉 Monitor，拔 USB，才移動線。

先把 DHT 的 GND 從 c3 移到 b3、VCC 從 c6 移到 b6。同列仍相通，只是讓出插孔。

元件	GND / 電源	訊號
ESP32	GND → a3 ; 3V3 → a6	板子留在桌面
DHT11	GND → b3 ; VCC → b6	DATA → 自己的 PIN_DHT
已確認的 KY-018	– → c3 ; 中間排針 → c6	S → a15 ; c15 → PIN_LIGHT



PIN_LIGHT 沿用上週這片板已確認的 ADC 腳位；與 PIN_DHT 不同。若採上週 GPIO4 接法，就接 GPIO4，不是接 GPIO15。

第 15 列只是麵包板的孔列編號。這裡同列 a15、c15 相通；它沒有自動連到晶片的 GPIO15。

換成整合程式，先選 1

先不判斷遮光，不接蜂鳴器。

開啟 [雙感測器程式](#)，完整版本見第 26 頁。不要把兩支程式直接貼在一起。

設定名稱	第一輪內容
LESSON_STAGE	1，這一輪只讀取資料
PIN_LIGHT、PIN_DHT	剛才接線的兩個不同 GPIO
SENSOR_PROFILES_CONFIRMED	兩個模組的本次接法確認後填 true
DEVICE_ID	自己組別；和前一支程式一致
四個 INDOOR / SHADE 數值	先保留 -1，這輪不用分類
所有 BUZZER 設定	保留預設，蜂鳴器不接

USB 拔除時，把 ESP32 上所有杜邦線拔下，包含 3V3、GND 與兩條訊號線；麵包板上的模組可原位保留。只接 USB、Verify、Upload；完成後拔 USB，再依前頁接回四條板端連線，核對後接 USB。

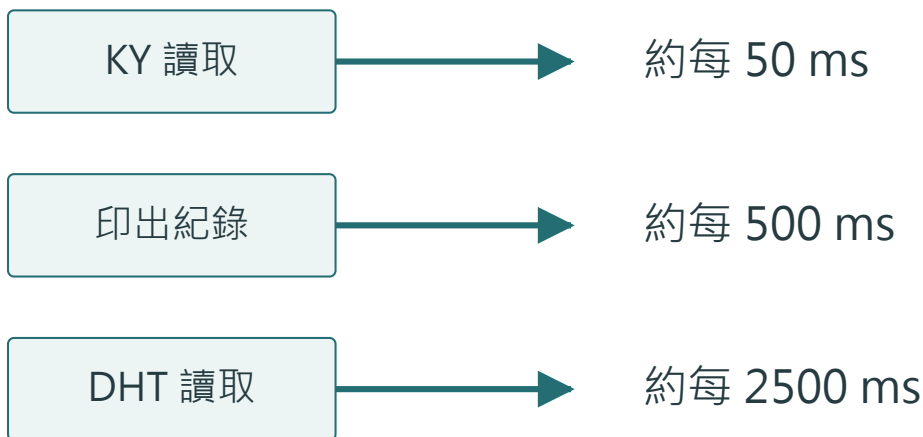
Serial Monitor 選 115200。若畫面說 sensor 設定未完成，回查本表，不把所有確認值一律改 true。

光線會變，溫濕度慢慢更新

這次看的是同一支程式的兩種讀取速度。

1. 保持室內光，保存幾行輸出。
2. 用手遮住 KY，再移開。只遮感光面，不碰金屬排針。
3. 比較 `light_raw`，確認它會改變；不預設遮住一定變大。
4. 等 DHT 更新，找出它的有效狀態與資料時間。

安排の間隔，非精準時間保證



KY 大約每 50 ms 讀一次，但一般紀錄約每 500 ms 印一次。`sensor=ky018` 這行的 `sample` 跳約 10，不是證明遺失十筆。

DHT 大約每 2500 ms 才讀新資料。中間重複列出上一筆 DHT 結果，是保留最近資訊，不是重新量了十次。

第一輪沒有分類，因此看到 `state=UNKNOWN`、`event_count=0` 是預期行為。此時先確認資料讀得到，不急著改門檻。

week04_dual_sensor_alarm.ino

第 1 ~ 32 行。

```
#include <Arduino.h>
#include <DHT.h>
#include <math.h>

// Published defaults are deliberately blocked. No inferred module pin order.
// 1: dual readings only; 2: stable cover events; 3: add the approved buzzer.
const int LESSON_STAGE = 3;
const int PIN_LIGHT = -1;
const int PIN_DHT = -1;
const int PIN_BUZZER_CONTROL = -1;
const bool SENSOR_PROFILES_CONFIRMED = false;
const bool BUZZER_PROFILE_CONFIRMED = false;
const int BUZZER_ON_LEVEL = -1; // Set HIGH or LOW only from the approved driver
profile.
const bool BUZZER_USE_TONE = false;
const int BUZZER_SERIES_OHMS = -1; // HW-508 tone path requires a verified 1 kOhm
resistor.
const uint32_t BUZZER_HZ = 2000;
const int INDOOR_MIN = -1;
const int INDOOR_MAX = -1;
const int SHADE_MIN = -1;
const int SHADE_MAX = -1;
const char *DEVICE_ID = "CHANGE_ME";
const uint32_t LIGHT_INTERVAL_MS = 50;
const uint32_t DHT_INTERVAL_MS = 2500;
const uint32_t STABLE_MS = 150;
const uint32_t BEEP_MS = 120;
const uint32_t PRINT_INTERVAL_MS = 500;

DHT dht(PIN_DHT, DHT11);
bool ready = false, shadeHigher = true;
int threshold = -1;
bool injectDht = false, injectLight = false;
bool muted = false, beepActive = false;
```

同一份 .ino 檔案；分頁不代表另開程式。

week04_dual_sensor_alarm.ino

第 33 ~ 65 行。

```
bool buzzerFault = false;
uint32_t beepStarted = 0, lastLight = 0, lastDht = 0, lastPrint = 0;
uint32_t lightSample = 0, dhtSample = 0, coverEvents = 0;
int lightRaw = -1;
bool lightValid = false;
// -1: no accepted stable classification yet; 0: indoor; 1: covered.
int candidate = -1, stable = -1;
uint32_t candidateSince = 0;
bool armed = false;
float temperatureC = NAN, humidityPct = NAN;
bool haveDht = false, dhtValid = false, previousDhtValid = false;
float previousC = 0, previousRh = 0;
uint32_t previousDhtMs = 0, dhtAt = 0;
const char *dhtQuality = "invalid", *dhtReason = "not_sampled";

bool configureCalibration() {
    if (INDOOR_MIN <= 0 || SHADE_MIN <= 0 || INDOOR_MAX >= 4095 || SHADE_MAX >= 4095 ||
        INDOOR_MIN > INDOOR_MAX || SHADE_MIN > SHADE_MAX) return false;
    if (INDOOR_MAX < SHADE_MIN) {
        shadeHigher = true; threshold = (INDOOR_MAX + SHADE_MIN) / 2;
    } else if (SHADE_MAX < INDOOR_MIN) {
        shadeHigher = false; threshold = (SHADE_MAX + INDOOR_MIN) / 2;
    } else return false;
    return true;
}

bool profileReady() {
    if (LESSON_STAGE < 1 || LESSON_STAGE > 3 || !SENSOR_PROFILES_CONFIRMED ||
        PIN_LIGHT < 0 || PIN_DHT < 0 || PIN_LIGHT == PIN_DHT) return false;
    if (LESSON_STAGE < 3) return true; // Unconnected buzzer is never accessed.
    return BUZZER_PROFILE_CONFIRMED && PIN_BUZZER_CONTROL >= 0 &&
        PIN_LIGHT != PIN_BUZZER_CONTROL && PIN_DHT != PIN_BUZZER_CONTROL &&
        (BUZZER_USE_TONE ? BUZZER_SERIES_OHMS == 1000 :
```

同一份 .ino 檔案；分頁不代表另開程式。

week04_dual_sensor_alarm.ino

第 66 ~ 95 行。

```
        (BUZZER_ON_LEVEL == HIGH || BUZZER_ON_LEVEL == LOW));
    }

    void silence() {
        if (LESSON_STAGE < 3) { beepActive = false; return; }
        if (BUZZER_USE_TONE) {
            if (!ledcWrite(PIN_BUZZER_CONTROL, 0)) {
                ledcDetach(PIN_BUZZER_CONTROL);
                digitalWrite(PIN_BUZZER_CONTROL, LOW); pinMode(PIN_BUZZER_CONTROL, OUTPUT);
                if (!buzzerFault) Serial.println("event_type=buzzer_fault reason=tone_stop_failed
restart_required=true");
                buzzerFault = true; muted = true;
            }
        } else digitalWrite(PIN_BUZZER_CONTROL, BUZZER_ON_LEVEL == HIGH ? LOW : HIGH);
        beepActive = false;
    }

    bool initializeBuzzer() {
        if (LESSON_STAGE < 3) return true;
        digitalWrite(PIN_BUZZER_CONTROL, BUZZER_USE_TONE ? LOW : 1-BUZZER_ON_LEVEL);
        pinMode(PIN_BUZZER_CONTROL, OUTPUT);
        if (BUZZER_USE_TONE && !ledcAttach(PIN_BUZZER_CONTROL, BUZZER_HZ, 10)) return false;
        silence();
        return !buzzerFault;
    }

    void startBeep(uint32_t now) {
        if (LESSON_STAGE < 3 || buzzerFault) return;
        if (BUZZER_USE_TONE) {
            if (ledcWriteTone(PIN_BUZZER_CONTROL, BUZZER_HZ) == 0) {
                buzzerFault = true; muted = true; silence();
            }
        }
    }
}
```

同一份 .ino 檔案；分頁不代表另開程式。

week04_dual_sensor_alarm.ino

第 96 ~ 125 行。

```
        Serial.println("event_type=buzzer_fault reason=tone_start_failed
restart_required=true");
        return;
    }
} else digitalWrite(PIN_BUZZER_CONTROL, BUZZER_ON_LEVEL);
beepActive = true; beepStarted = now;
}

void serviceBeep(uint32_t now) {
    if (beepActive && now - beepStarted >= BEEP_MS) {
        silence();
        Serial.printf("device_id=%s event_type=beep_off uptime_ms=%lu
reason=duration_elapsed\n", DEVICE_ID, (unsigned long)now);
    }
}

void resetLightState() {
    candidate = stable = -1;
    armed = false; // A new stable release is required after any invalid interval.
}

void readLight(uint32_t now) {
    ++lightSample;
    lightRaw = injectLight ? -1 : analogRead(PIN_LIGHT);
    if (LESSON_STAGE == 1) {
        // Range check only: this stage makes no indoor/shade or event claim.
        lightValid = !injectLight && lightRaw > 0 && lightRaw < 4095;
        resetLightState();
        return;
    }
    // Controller policy is stricter than Week 3's candidate label: outside observed span
    is rejected.
    lightValid = !injectLight && lightRaw > 0 && lightRaw < 4095 &&
```

同一份 .ino 檔案；分頁不代表另開程式。

week04_dual_sensor_alarm.ino

第 126 ~ 153 行。

```

        lightRaw >= min(INDOOR_MIN, SHADE_MIN) && lightRaw <= max(INDOOR_MAX,
SHADE_MAX);
    if (!lightValid) { resetLightState(); silence(); return; }
    const bool lowSide = lightRaw <= threshold;
    const int current = (shadeHigher ? !lowSide : lowSide) ? 1 : 0;
    if (candidate != current) { candidate = current; candidateSince = now; }
    if (now - candidateSince < STABLE_MS || stable == candidate) return;
    stable = candidate;
    Serial.printf("device_id=%s event_type=light_stable state=%s uptime_ms=%lu
source=hardware\n",
        DEVICE_ID, stable ? "SHADE" : "INDOOR", (unsigned long)now);
    if (stable == 0) { armed = true; return; }
    if (!armed) return; // Booting while covered is not a new cover action.
    armed = false;
    ++coverEvents;
    Serial.printf("device_id=%s event_type=cover_event count=%lu uptime_ms=%lu
buzzer_command=%s\n",
        DEVICE_ID, (unsigned long)coverEvents, (unsigned long)now,
        LESSON_STAGE < 3 ? "disabled" : muted ? "muted" : "on");
    if (!muted) {
        startBeep(now);
    }
}

void readDht(uint32_t now) {
    (void)now; // Acquisition timestamp is taken after the synchronous library call.
    ++dhtSample;
    temperatureC = humidityPct = NAN;
    if (!injectDht) { humidityPct = dht.readHumidity(); temperatureC =
dht.readTemperature(); }
    dhtAt = millis(); // Acquisition completion time; not the KY sample's timestamp.
    haveDht = true; dhtValid = true;
}

```

同一份 .ino 檔案；分頁不代表另開程式。

week04_dual_sensor_alarm.ino

第 154 ~ 179 行。

```
dhtQuality = "usable"; dhtReason = "basic_checks_passed";
if (!isfinite(temperatureC) || !isfinite(humidityPct)) {
    dhtValid = false; dhtQuality = "invalid";
    dhtReason = injectDht ? "injected_read_failed" : "read_failed";
} else if (humidityPct < 0 || humidityPct > 100) {
    dhtValid = false; dhtQuality = "invalid"; dhtReason = "rh_out_of_bounds";
} else if (temperatureC < 10 || temperatureC > 40) {
    dhtQuality = "suspect"; dhtReason = "outside_classroom_policy";
} else if (previousDhtValid && dhtAt - previousDhtMs <= 2 * DHT_INTERVAL_MS &&
    (fabsf(temperatureC - previousC) > 5 || fabsf(humidityPct - previousRh) >
10)) {
    dhtQuality = "suspect"; dhtReason = "abrupt_change";
}
previousDhtValid = dhtValid && !injectDht;
if (previousDhtValid) { previousC=temperatureC; previousRh=humidityPct;
previousDhtMs=dhtAt; }
// Report each sensor's own acquisition, rather than faking simultaneous samples.
Serial.printf("device_id=%s sensor=dht11 source=%s sample=%lu uptime_ms=%lu
temperature_c=%.1f humidity_pct=%.1f valid=%s quality=%s reason=%s\n",
    DEVICE_ID, injectDht ? "injected" : "hardware", (unsigned
long)dhtSample,
    (unsigned long)dhtAt, temperatureC, humidityPct, dhtValid ? "true" :
"false", dhtQuality, dhtReason);
}

void setup() {
    Serial.begin(115200); delay(500);
    if (!profileReady() || (LESSON_STAGE >= 2 && !configureCalibration())) {
        Serial.println("week=4 status=blocked reason=profile_or_calibration_missing");
    }
    return;
}
// The approved module must also remain quiet before setup and during reset.
```

同一份 .ino 檔案；分頁不代表另開程式。

week04_dual_sensor_alarm.ino

第 180 ~ 205 行。

```

    if (!initializeBuzzer()) {
        Serial.println("week=4 status=blocked reason=buzzer_init_failed"); return;
    }
    analogReadResolution(12); analogSetPinAttenuation(PIN_LIGHT, ADC_11db);
    dht.begin();
    lastLight = lastDht = lastPrint = millis(); ready = true;
    Serial.printf("lesson_stage=%d buzzer_enabled=%s\n", LESSON_STAGE, LESSON_STAGE == 3
? "true" : "false");
    Serial.printf("week=4 status=ready device_id=%s threshold=%d light_interval_ms=%lu
dht_interval_ms=%lu stable_ms=%lu beep_ms=%lu\n",
        DEVICE_ID, threshold, (unsigned long)LIGHT_INTERVAL_MS, (unsigned
long)DHT_INTERVAL_MS,
        (unsigned long)STABLE_MS, (unsigned long)BEEP_MS);
    Serial.println("commands=f:inject_dht k:inject_light r:resume_both q:mute u:unmute");
}

void loop() {
    if (!ready) return;
    uint32_t now = millis();
    serviceBeep(now);
    if (Serial.available()) {
        const char c=Serial.read();
        if (c=='f' || c=='r') { injectDht=c=='f'; previousDhtValid=false; haveDht=false;
dhtValid=false; temperatureC=humidityPct=NAN; lastDht=now; }
        if (c=='k' || c=='r') { injectLight=c=='k'; lightRaw=-1; lightValid=false;
resetLightState(); silence(); }
        if (c=='q') { muted=true; silence(); }
        if (c=='u' && !buzzerFault) muted=false; // Does not replay a previous event.
        if (c!='\n' && c!='\r' && c!=' ') {
            const bool known=c=='f' || c=='k' || c=='r' || c=='q' || (c=='u' && !buzzerFault);
            Serial.printf("device_id=%s event_type=command status=%s command=%c
uptime_ms=%lu\n", DEVICE_ID, known?"accepted":"rejected", c, (unsigned long)now);

```

同一份 .ino 檔案；分頁不代表另開程式。

week04_dual_sensor_alarm.ino

第 206 ~ 223 行。

```
    }  
  }  
  if (now-lastLight >= LIGHT_INTERVAL_MS) { lastLight=now; readLight(now); }  
  // Avoid a DHT transaction while a beep is active, so it cannot lengthen this pulse.  
  if (!beepActive && now-lastDht >= DHT_INTERVAL_MS) { lastDht=now; readDht(now); }  
  now=millis(); serviceBeep(now);  
  if (now-lastPrint >= PRINT_INTERVAL_MS) {  
    lastPrint=now;  
    Serial.printf("device_id=%s sensor=ky018 source=%s sample=%lu uptime_ms=%lu  
age_ms=%lu light_raw=%d valid=%s state=%s candidate=%d armed=%s event_count=%lu\n",  
                  DEVICE_ID, injectLight?"injected":"hardware", (unsigned  
long)lightSample, (unsigned long)lastLight, (unsigned long)(now-lastLight),  
                  lightRaw, lightValid?"true":"false",  
stable<0?"UNKNOWN":stable?"SHADE":"INDOOR", candidate,  
                  armed?"true":"false", (unsigned long)coverEvents);  
    if (!haveDht) Serial.println("sensor=dht11 snapshot=none valid=false  
age_ms=unknown");  
    else Serial.printf("sensor=dht11 snapshot=latest source=%s age_ms=%lu valid=%s  
quality=%s reason=%s\n",  
                      injectDht?"injected":"hardware", (unsigned long)(now-dhtAt),  
                      dhtValid?"true":"false", dhtQuality, dhtReason);  
  }  
}
```

同一份 .ino 檔案；分頁不代表另開程式。

同次輸出，不等於同時測量

程式會分行列出 KY 資料和最近的 DHT 狀態。

教學假資料： 整合程式於開機後 3025 ms 完成一次 DHT 讀取，3500 ms 印出紀錄。

```
DHT 資料距今時間 = 現在時間 - 讀取完成時間
                  = 3500 - 3025
                  = 475 ms
```

所以光線可以較新，溫濕度則已存在約 475 ms。找 `sensor=dht11 snapshot=latest` 這行的 `age_ms`，不要和 KY 那行的 `age_ms` 混用。

看見什麼	不能誤認為什麼
尚未完成第一筆 DHT 讀取	溫度就是 0°C
最近一筆 DHT 讀取失敗	舊正常值可以改標為最新有效值
相鄰兩行有兩個感測器	兩個感測器在同一瞬間完成測量

單獨 DHT 程式的 `uptime_ms` 在開始讀取時記錄；整合程式另記 DHT 完成時間。比較不同程式的紀錄時，要看欄位定義，不只看名字像不像。

沿用上週基準，加入遮光判斷

接線不變，蜂鳴器仍不接。

將 LESSON_STAGE 改成 2，填入上週同一組接法與環境的四個值：

設定	自己的紀錄
INDOOR_MIN	一般室內光最小 raw：_ _
INDOOR_MAX	一般室內光最大 raw：_ _
SHADE_MIN	遮光最小 raw：_ _
SHADE_MAX	遮光最大 raw：_ _

兩段範圍不能重疊或碰在一起，原始值也不應為 0 或 4095。程式會依兩段的方向選擇判斷方式，不固定遮光比較大。

Verify、Upload 後，看開頭是否說基準可用。若不能判斷，先查四個值與本次接法，不任意填示例的 300、900。

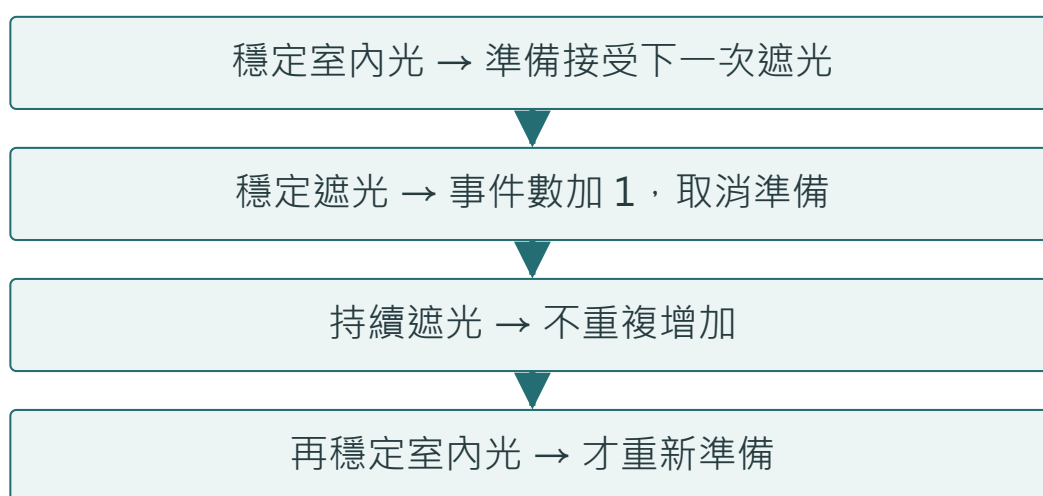
若換模組、位置、供電或照明，上週基準可能不能沿用。回到 Week 3 的基準建立步驟，只補受影響的紀錄；不混合不同次測量湊成一組。

本輪上傳只改分類設定，已確認的感測線可維持原接法；要移線仍先拔 USB。保留蜂鳴器設定 false / -1。

放開，遮住，再放開

看事件數，不只是看某一次 raw。

1. 先讓 KY 在室內光下穩定至少約 0.3 秒。
2. 遮住至少約 0.3 秒，查看事件數是否增加 1。
3. 持續遮住約 2 秒，確認不一直加次數。
4. 放開至少約 0.3 秒，再遮住。這時才應再增加 1。



剛開機就一直遮住，不應立即算一次新事件。要先有穩定的室內光，才有下一次「從放開到遮住」。

保存同一次測試的紀錄	自己填
操作前事件數	--
第一次遮住 / 持續蓋住	-- / --
放開，再次遮住	--

事件數增加不代表蜂鳴器已發聲。本輪本來就是無聲測試。

為什麼不碰到門檻就立刻算？

手晃一下、光線抖一下，不一定是一次完整動作。

程式先得到暫時分類 `candidate`。連續取樣得到同一分類，持續至少 **150 ms**，才更新穩定分類 `stable`。

假設四次取樣都暫分為遮光



`raw` 可以是 905、908、902、910。

中途分類不同，就重新計時。

這是**理想取樣時間示例**。確認的是分類維持一致，不是要求 `raw` 每次完全相等。

約 50 ms 是安排的取樣間隔；程式執行與感測器讀取也需要時間，實際事件不保證剛好在第 150 ms 發生。

穩定分類用來減少很短的變化造成重複事件，**不會把錯接的感測器變正確，也不證明距離或亮度準確**。

數字穩定，仍要看它從哪裡來

Week 4 的事件判斷和 Week 3 的品質標記並非完全相同。

教學假基準：室內光 300 ~ 320，遮光 900 ~ 920，分界為 610。

新 raw	Week 4 整合程式怎麼處理
310	在整體範圍內，暫分室內光
910	在整體範圍內，暫分遮光
700	仍在 300 ~ 920 內，暫分遮光，但不在兩段實測基準內
1000、0、4095	不接受本次光線分類，清除穩定狀態

因此，700 維持足夠久，程式仍可能算遮光事件。**基準中沒有直接觀察過這一段**，不要把它寫成同等充分的判斷證據。

Week 3 已練過保留原值與品質原因。今天也要留下 raw，遇到這類中間值時，先核對光線位置與基準，不能只交一個「成功」標籤。

如果光線資料無效，程式會取消已準備好的遮光事件；恢復後要先穩定放開，再遮住。

DHT 失敗，和光敏失敗不同

仍在第 2 輪，先不用聲音判斷成功。

送出的字母	發生什麼	應觀察什麼
f	模擬 DHT 無效資料	DHT 無效；光線仍可產生事件
k	模擬 KY 無效 raw	光線狀態清空，不產生新遮光事件
r	兩者恢復實際讀取	等新資料；先放開，再遮住

先測 $f \rightarrow r$ ，保存紀錄；再測 $k \rightarrow r$ 。不要 f 、 k 一起送，否則難以分辨改變來自哪一個。

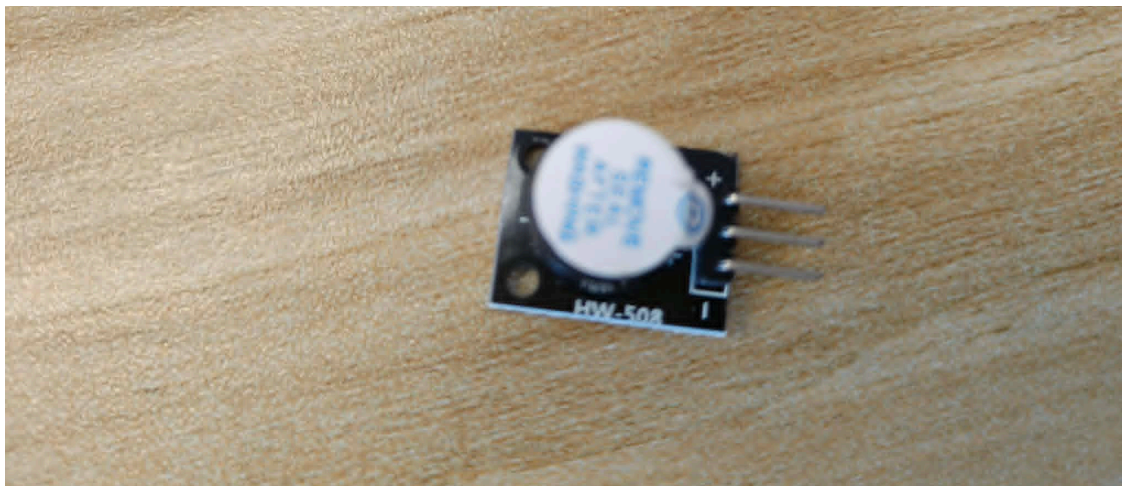
用一行文字記錄每次操作，例如「送 k 後持續遮住，事件數仍是 2； r 後放開再遮住，才變 3」。填自己的數字。

這支程式的選擇： DHT 只負責環境紀錄，失敗不阻止光敏事件。KY 才是這個遮光動作的依據。

如果未來作品是依溫度控制加熱，DHT 失敗的處理就不能照搬。本課不是示範加熱器控制，也沒有接危險負載。

先認出自己的蜂鳴器

商品名稱相同，不代表接法與驅動方式相同。



已購 HW-508 的實物。不能只看「有源蜂鳴器」商品名稱就決定接 3.3 V 或 GPIO。

前面作品到這裡已能記錄環境與遮光事件。只有本批蜂鳴器的接線、電流及停止方式確認後，才做聲音部分。

課堂提供的接法	程式需要的內容
已確認的有效準位控制模組	控制 GPIO、HIGH 或 LOW 有效、獨立供電 / 共地接法
已確認的 HW-508 受限波形接法	控制 GPIO、串聯 1 k Ω 、2000 Hz 波形，其他腳依確認圖

兩種不能混用。不能把「有三支腳」一律當 VCC、GND、S，也不能用 220 Ω 取代指定的 1 k Ω 。

不把蜂鳴器直接跨接 3V3 與 GND 當成通用試法。未完成確認時保持無聲版本，先用事件紀錄完成操作與練習，不自行試插未知腳位。

把一千歐姆放在訊號路徑中

下表把 T01 已有「有聲」觀察的功能接法，整理成這次麵包板孔位。新孔位並非新增實測；限同型 HW-508，仍須課前確認電流、上電與停止條件。尚未確認就保留無聲版。

先讀下一頁設定並完成裸板上傳，再拔 USB、確認 PWR 熄滅，依序接：

元件 / 導線	起點 → 終點
控制線	GPIO18 → a10，前提是未與 DHT / KY 共用
1 kΩ 電阻	c10 → c12，不可兩腳插同列
蜂鳴器正端	HW-508 印有 + 的腳 → e12
蜂鳴器負端	印有 - 的腳 → d3；a3 已接板 GND
中間腳	不接，與其他金屬接點保持分離

GPIO18 → a10 / c10 → 1 kΩ → c12 / e12 → HW-508 +
HW-508 - → d3 / a3 → 板 GND

第 10、12 列沒有直接導線相連；只有電阻跨接。這裡不是把蜂鳴器直接接在 3V3 與 GND。依板上 + / - 絲印找腳，不用照片左右方向猜。

對應設定：PIN_BUZZER_CONTROL=18、BUZZER_USE_TONE=true、BUZZER_SERIES_OHMS=1000；保留 BUZZER_HZ=2000。波形模式的 BUZZER_ON_LEVEL 不使用，可保持 -1。確認旗標仍依實物條件，不因填完這些值就自動通過。

用同一個事件觸發短音

先上傳能保持安靜的設定，再斷電接線。

維持感測器設定，改 `LESSON_STAGE = 3`。

設定	依本批確認結果填寫
<code>PIN_BUZZER_CONTROL</code>	第三個不同 GPIO，不共用 KY 或 DHT
<code>BUZZER_PROFILE_CONFIRMED</code>	確認完成才 true
<code>BUZZER_USE_TONE</code>	HW-508 受限波形方式 true；有效準位方式 false
<code>BUZZER_SERIES_OHMS</code>	受限波形方式必須是 1000，且實際串入 1 k Ω
<code>BUZZER_ON_LEVEL</code>	有效準位方式填已確認的 HIGH 或 LOW；不能猜

1. 蜂鳴器仍不接，Verify、Upload。
2. 關掉 Monitor、拔 USB、PWR 熄滅；相符 HW-508 依前頁表接蜂鳴器，其他型式不能套用。感測器線不移動。
3. 再次核對後通電，先看上電時是否安靜。
4. 放開後遮住，應增加一次事件，並短叫一次。

持續叫、發熱、重新開機，或聲音和命令不一致，先拔 USB。不要靠一直重啟或改大音量繼續測試。

q 是靜音，不是切斷電源

分別確認事件、程式命令與實際聲音。

1. 送 **q**。若正在叫，程式應停止聲音；後續遮光仍記事件，但不叫。
2. 放開，再遮住，確認事件數增加、蜂鳴器安靜。
3. 送 **u** 解除靜音。它不補播剛才的事件。
4. 再放開、再遮住，才應有新的短音。

光線事件 → 程式發出開始聲音命令 → 蜂鳴器
時間到 / **q** → 程式發出停止命令 → 檢查是否真的安靜

本例聲音目標約 **120 ms**。軟體目標不等於已量到精確聲長；本週不要求靠耳朵判斷毫秒精度。

`beep_off` 只能證明程式走到停止命令，不能代替耳朵觀察或硬體量測。持續叫就斷電排查。

若波形輸出初始化失敗，程式會保持故障與靜音，**u** 不能強行解除。修正原因後重新啟動再測。

幫另一組判讀測試紀錄

以下是假想操作；假設光線讀值有效，時間都超過穩定門檻。

重新開機，事件數從 0 開始：

操作順序	請填事件總數
一開始就遮住 2 秒	A： _ _
放開 0.3 秒，再遮住 0.3 秒	B： _ _
持續遮住 2 秒	C： _ _
送 q，放開再遮住	D： _ _ ；會不會叫？ _ _
送 u，但手仍遮住	E： _ _ ；會不會補叫？ _ _

再回答：

1. 最近 DHT 結果為 NaN，可以把上一筆 25°C 改標成這次有效值嗎？
2. 在 3500 ms 印紀錄，DHT 讀取完成時間為 3025 ms，資料距今多久？
3. 程式印 beep_off，但實物還在叫，下一個動作是什麼？

每題寫出事件數、時間或停止動作，並說明理由。

做一個按鈕式環境紀錄器

沿用 Week 3 的「按一下才記錄」，這次把溫濕度也放進同一行。

已教過的零件	本例用途
Week 2 按鈕	決定何時建立一筆紀錄
Week 3 KY-018	按下時取得光線 raw
Week 4 DHT11	定時讀取，提供最近一次溫濕度與時間

例如把紙盒當展示櫃模型，在「盒蓋開啟」「盒蓋關閉」時各按一次，旁邊寫下操作條件。這不是保存食品的安全監控器。

先預測：剛按完又按一次，溫濕度會不會每次都是新量的？答案要看時間欄，不是看溫度有沒有變。

本例只接按鈕、光敏與 DHT；斷電後移除蜂鳴器。

三個訊號，不能共用一個 GPIO

先拔 USB、拆除外接線，開啟 [完整環境紀錄程式](#)。確認下列設定後只接 USB 上傳，再拔 USB 接線。

功能	接線與程式設定
GND	板 GND → a3 ; KY – → b3 ; DHT GND → c3 ; a29 → e3
3V3	板 3V3 → a6 ; KY 電源 → b6 ; DHT VCC → c6
KY 訊號	S → a15 ; c15 → PIN_LIGHT，本稿確認後用 GPIO4
按鈕訊號	GPIO5 → a27，PIN_BUTTON = 5，限相符且已確認的板卡
DHT 訊號	DATA → PIN_DHT，填前面已驗證且不同於 4 / 5 的腳位

按鈕四腳插 **e27**、**f27**、**e29**、**f29**，沿用 Week 2 的方向，跨中央溝槽。斷電量 b27 對 b29，應放開不通、按下才通。訊號走 GPIO5 → a27 → e27；按下後經 e29 → a29 → e3 回地。共地不是把三條訊號接在一起。

只在本組腳位、模組腳序、3.3 V 供電及訊號確認後，才把 PROFILE_CONFIRMED 改 true。不知道 DHT 腳位時回看本組前段紀錄，不猜 6 或 7。

先等三秒，再按一下

接 USB，Monitor 選 115200。放開按鈕至少 0.1 秒，等出現 `event=dht_attempt`，再按住半秒。

以下為分行顯示的**教學假資料**，實際輸出在同一行：

```
event=record attempt=1 record=1 raw=420 endpoint=0
temperature_c=25.0 humidity_pct=50.0 dht_valid=1
light_uptime_ms=3500 dht_read_finished_ms=3025 dht_age_ms=475
```

接著操作	觀察
按住兩秒	不應一直新增 record
放開，再按	attempt 和 record 各加一
在下一次 DHT 讀取前再次按	光線新讀；DHT 仍可能是上一筆，age 變大
Reset 後很快按	尚未讀 DHT 時，溫濕度為 nan、valid=0、age=NA

本例每 2.5 秒嘗試 DHT，讀取可能短暫占用程式。非常短的按壓可能漏掉，不能當成精準計次儀器。資料不會自動寫入硬碟，保存 Monitor 紀錄才留下證據。

button_environment_log.ino

第 1 ~ 33 行。

```
#include <Arduino.h>
#include <DHT.h>
#include <math.h>

const bool PROFILE_CONFIRMED = false;
const int PIN_LIGHT = -1;
const int PIN_BUTTON = -1;
const int PIN_DHT = -1;
const uint32_t DHT_INTERVAL_MS = 2500;
const uint32_t DEBOUNCE_MS = 40;
DHT dht(PIN_DHT, DHT11);

bool ready = false, armed = false, haveDht = false, dhtValid = false;
int lastButton = HIGH;
float temperature = NAN, humidity = NAN;
uint32_t changedAt = 0, dhtScheduledAt = 0, dhtFinishedAt = 0;
uint32_t attempt = 0, recordCount = 0;

bool pressed(uint32_t now) {
    const int level = digitalRead(PIN_BUTTON);
    if (level != lastButton) { lastButton = level; changedAt = now; }
    if (uint32_t(now - changedAt) < DEBOUNCE_MS) return false;
    if (level == HIGH) { armed = true; return false; }
    if (!armed) return false;
    armed = false;
    return true;
}

void saveRecord(uint32_t now) {
    ++attempt;
    ++recordCount; // Basic example records the latest result, including invalid data.
    const int raw = analogRead(PIN_LIGHT);
    Serial.printf("event=record attempt=%lu record=%lu raw=%d "

```

同一份 .ino 檔案；分頁不代表另開程式。

button_environment_log.ino

第 34 ~ 67 行。

```
        "endpoint=%d temperature_c=%.1f humidity_pct=%.1f "
        "dht_valid=%d light_uptime_ms=%lu ",
        (unsigned long)attempt, (unsigned long)recordCount,
        raw, raw == 0 || raw == 4095, temperature, humidity,
        dhtValid, (unsigned long)now);
    if (haveDht)
        Serial.printf("dht_read_finished_ms=%lu dht_age_ms=%lu\n",
            (unsigned long)dhtFinishedAt,
            (unsigned long)uint32_t(now - dhtFinishedAt));
    else Serial.println("dht_age_ms=NA reason=not_read_yet");
}

void setup() {
    Serial.begin(115200);
    if (!PROFILE_CONFIRMED || PIN_LIGHT < 0 || PIN_BUTTON < 0 ||
        PIN_DHT < 0 || PIN_LIGHT == PIN_BUTTON || PIN_LIGHT == PIN_DHT ||
        PIN_BUTTON == PIN_DHT) {
        Serial.println("event=blocked reason=check_configuration");
        return;
    }
    pinMode(PIN_BUTTON, INPUT_PULLUP);
    analogReadResolution(12);
    analogSetPinAttenuation(PIN_LIGHT, ADC_11db);
    dht.begin();
    changedAt = dhtScheduledAt = millis();
    ready = true;
    Serial.println("event=ready release_button_first");
}

void loop() {
    if (!ready) return;
    uint32_t now = millis();
    if (uint32_t(now - dhtScheduledAt) >= DHT_INTERVAL_MS) {
        dhtScheduledAt = now;
```

同一份 .ino 檔案；分頁不代表另開程式。

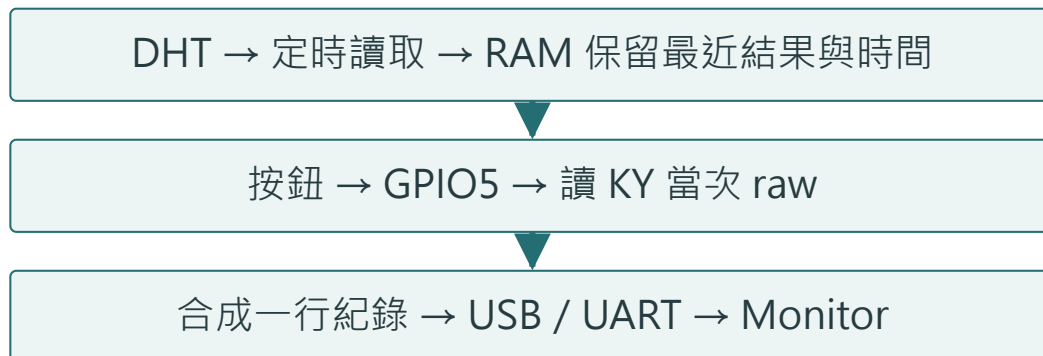
button_environment_log.ino

第 68 ~ 80 行。

```
temperature = dht.readTemperature();
humidity = dht.readHumidity();
dhtFinishedAt = millis();
haveDht = true;
dhtValid = isfinite(temperature) && isfinite(humidity) &&
            temperature >= 0 && temperature <= 50 &&
            humidity >= 0 && humidity <= 100;
Serial.printf("event=dht_attempt valid=%d finished_ms=%lu\n",
              dhtValid, (unsigned long)dhtFinishedAt);
}
now = millis();
if (pressed(now)) saveRecord(now);
}
```

同一份 .ino 檔案；分頁不代表另開程式。

一筆紀錄，不代表同時測量



按下時，程式立刻讀 KY，並附上最近一次 DHT 讀取結果。這樣不必每按一次就要求 DHT 重新讀。

按鈕電流：3.3 V → 內部上拉電阻 → GPIO5 → 按鈕 → GND
模組供電：3V3 → KY / DHT 各自的電路 → GND → 板上電源
資訊：KY S → ADC；DHT DATA ↔ 程式庫 → RAM 中最近結果

`dht_age_ms` 是距離最近一次讀取完成多久，不是感測器內部精準取樣時間。`dht_valid=1` 只表示這次通過程式的基本數值檢查，不表示經過儀器校準。

若本次 DHT 失敗，就保留失敗；程式不把上一次成功的 25°C 偽裝成本次新資料。

時間戳改成現在，舊資料變新了嗎？

資訊處理示意；僅作圖上推演，保持現有實物接線不動。

正常的連接 / 處理



只改標記的地方



箭頭表示資訊處理順序，不是供電或電流路徑。

想一想：最後一次 DHT 完成於 3000 ms，按鈕在 4800 ms 按下。若把讀取時間改寫成 4800，會漏掉什麼？

原理提示：真正資料年齡是 1800 ms；只改時間欄會印成 0，但沒有發生新量測。

只有標記處改變，其餘供電、程式與環境條件沿用正常情境。不可把未知結果直接寫成 0、LOW 或「一定停止」。

展示櫃環境資料收錄

按下按鈕，表示想收錄一筆；不代表當下資料一定可用。

沿用剛才的按鈕、KY-018、DHT11 與基本程式。把紙盒當成展示櫃模型，每次開啟或關閉後按一下，留下環境紀錄。

這次只有符合以下條件才收錄：

檢查順序	不符合時的 reason
1. 已完成至少一次 DHT 讀取	not_read_yet
2. 最近一次 DHT 結果有效	dht_invalid
3. DHT 讀取完成距今不超過 1000 ms	dht_stale
4. 本次光線 raw 不是 0 或 4095	light_endpoint

1000 ms 是本題自訂的收錄規則，不是 DHT11 的規格。資料太舊不等於感測器故障；保持每 2500 ms 讀 DHT，不能為了過關而加快讀取。

每次有效按壓都重新讀取一次 KY，並只回報第一個不符合的原因。這不是食品或文物保存的安全監控器。

有成功，也要看得見拒絕

欄位	規則
attempt	每次有效按壓加 1
record	全部條件通過才加 1
skipped	任一條件不通過就加 1
reason	成功為 ok；失敗依檢查順序回報
raw、dht_age_ms	保留本次光線與 DHT 資料年齡；尚未讀過時年齡為 NA

每次按下只印一行 `event=record` 或 `event=skipped`。每一行都要滿足：

```
attempt = record + skipped
```

按住不放不重複新增。放開再按，才重新判斷；剛才被跳過的資料不得在背景自動補收錄。
Reset 後次數歸零。

沿用基本程式的按鈕辨識與定時 DHT 讀取。新增的是收錄條件與紀錄，不增加零件。

同樣按七次，只有兩次收錄

教學假資料；假設每列都是一次新的有效按下，從 Reset 後依序測試。

次序	按下時的資料	結果 / 原因	record / skipped
1	尚未讀 DHT， raw=420	skipped / not_read_yet	0 / 1
2	DHT 無效，raw=420	skipped / dht_invalid	0 / 2
3	DHT 有效、距今 1001 ms，raw=420	skipped / dht_stale	0 / 3
4	DHT 有效、距今 1000 ms，raw=0	skipped / light_endpoint	0 / 4
5	DHT 有效、距今 1000 ms，raw=420	record / ok	1 / 4
6	DHT 有效、距今 400 ms，raw=4095	skipped / light_endpoint	1 / 5
7	DHT 有效、距今 400 ms，raw=420	record / ok	2 / 5

第七列的預期輸出（其他感測數字依當次資料）：

```
event=record attempt=7 record=2 skipped=5 reason=ok  
raw=420 dht_age_ms=400
```

為方便閱讀分成兩行；程式應印在同一行。第 3 列即使 raw 同時為 0，也先回報 dht_stale。

等待，也會改變能否收錄

1. Reset 後放開按鈕，再於第一筆 DHT 完成前按一次，應跳過。
2. 等出現有效 `event=dht_attempt` 後立刻按一次；`raw` 不在端點時應收錄。
3. 放開按鈕，等到該筆 DHT 已超過一秒、下一筆尚未完成，再按一次，應因資料太舊跳過。
4. 等下一次有效 DHT 更新，放開再按，應恢復收錄。
5. 比較三個次數，確認每次嘗試都有成功或跳過的去向。

用紀錄中的時間核對，不只靠手按秒數。剛好 1000 ms 與 1001 ms 的邊界、無效值與端點，用上一頁假資料逐列走程式或用主機測試。

不要為了製造失敗拔帶電 DATA，也不要把 GPIO 短接到電源或地。 感測器沒有出現的條件，標成紙上判讀或模擬，不冒充實測。

保留一段能重現的操作紀錄

不另外增加一份重複報告，把資料放進原本的課堂筆記。

留下的內容	要能回答的問題
220、330 Ω 的讀值與量程	你量的是什麼？超量程和斷路怎麼分？
已確認腳位表與接線照片	這次用哪片板、哪個模組、接哪個 GPIO？
DHT 正常、f、r 的紀錄	是真實讀取還是模擬？恢復有證據嗎？
雙感測器與遮光操作紀錄	哪筆資料較舊？一次操作為何只計一次？
有聲版的 q / u；或註明未接蜂鳴器	軟體停止與實際安靜是否一致？

程式一併保留本次設定與程式庫版本。不要只留裁掉欄位名稱的螢幕截圖。

收尾：送 q (有聲版) → 關 Monitor → 拔 USB → 確認 PWR 熄滅 → 電表 OFF。拍照後再收線，電阻分袋標示。

下次可把同樣的資料顯示在作品上，或交給其他介面。先能說清楚資料從哪裡來、何時取得、失敗怎麼表示，再增加功能。

一次只排查一件事

看見的情況	下一個檢查
DHT.h 找不到	回程式庫管理員查 Adafruit DHT 安裝
無法上傳 / Monitor 空白	查 COM 接頭、資料線、實際埠號、115200；不要先改感測線
設定未完成或 blocked	查腳位保護值、確認欄位與本次第 1 / 2 / 3 輪設定
DHT 持續 read_failed	先記錄，拔 USB 後查腳序、GPIO、供電；不要自行升至 5 V
KY raw 不變或到端點	拔 USB，查 S → a15、c15 → GPIO，及電源 / 共地
沒有遮光事件	第 1 輪不分類；第 2 輪查四個基準，先放開再遮住
蜂鳴器沒叫	先查事件有無增加，再查是否 q、是否第 3 輪、故障紀錄；未知腳不可試插
蜂鳴器持續叫、發熱或板重啟	先拔 USB；保存現象後排查，不反覆通電硬試

查不到原因時留下：正在用的程式、設定、完整錯誤、接線照片、最後一個操作。不要只寫「不能用」。

色環、容差與量到的數字

先辨認四環或五環，再讀數字；不能把兩種規則混用。

環數	讀法	330 Ω、±5% 示例
四環	前兩環數字 × 第三環倍率；第四環容差	橙、橙、棕、金：33 × 10
五環	前三環數字 × 第四環倍率；第五環容差	橙、橙、黑、黑、金：330 × 1

黑 0、棕 1、紅 2、橙 3、黃 4、綠 5、藍 6、紫 7、灰 8、白 9。倍率另依色碼表查，容差也有自己的規則；金色容差為 ±5%。

教學示例：標稱 330 Ω、±5%，合適量程讀到 326 Ω。

$$\text{差異} = 326 - 330 = -4 \, \Omega$$

$$\text{差異幅度} = 4 \div 330 \times 100\% \approx 1.21\%$$

$$\text{標稱容差帶} = 330 \pm 16.5 \, \Omega = 313.5 \sim 346.5 \, \Omega$$

326 落在標稱容差帶內，但這不是精密認證。負的差值表示比標稱值小，不是負電阻；1.21% 也不是電表精度。

辨識困難時先查包裝、在斷電且隔離的條件量阻值，再核對色環，不只靠顏色猜。

查規格，要查對對象

想查的問題	原始資料
DHT 程式庫如何讀取、等待與回傳 NaN	Adafruit DHT sensor library 1.4.7
DHT 的基本連接與元件差異	Adafruit : Connecting to a DHTxx Sensor ；裸元件圖不是本批三線板的腳序
相對濕度的意思	香港天文台：Let's talk about relative humidity
電阻量測安全	Fluke 101 使用手冊，Measure Resistance ；用來核對斷電及表筆原則，不代替 A830L 的檔位說明
電阻色環	Vishay : Resistor Color Code
ESP32 波形輸出 API	Espressif : Arduino-ESP32 LEDC

三份完整程式分別在第 13、26、48 頁。未確認的腳位與供電設定保持停用。

上週 KY 基準仍可用就沿用；缺少基準時回 Week 3 補做，不因進入新一週而重收所有資料。